University of Nottingham
UK | CHINA | MALAYSIA

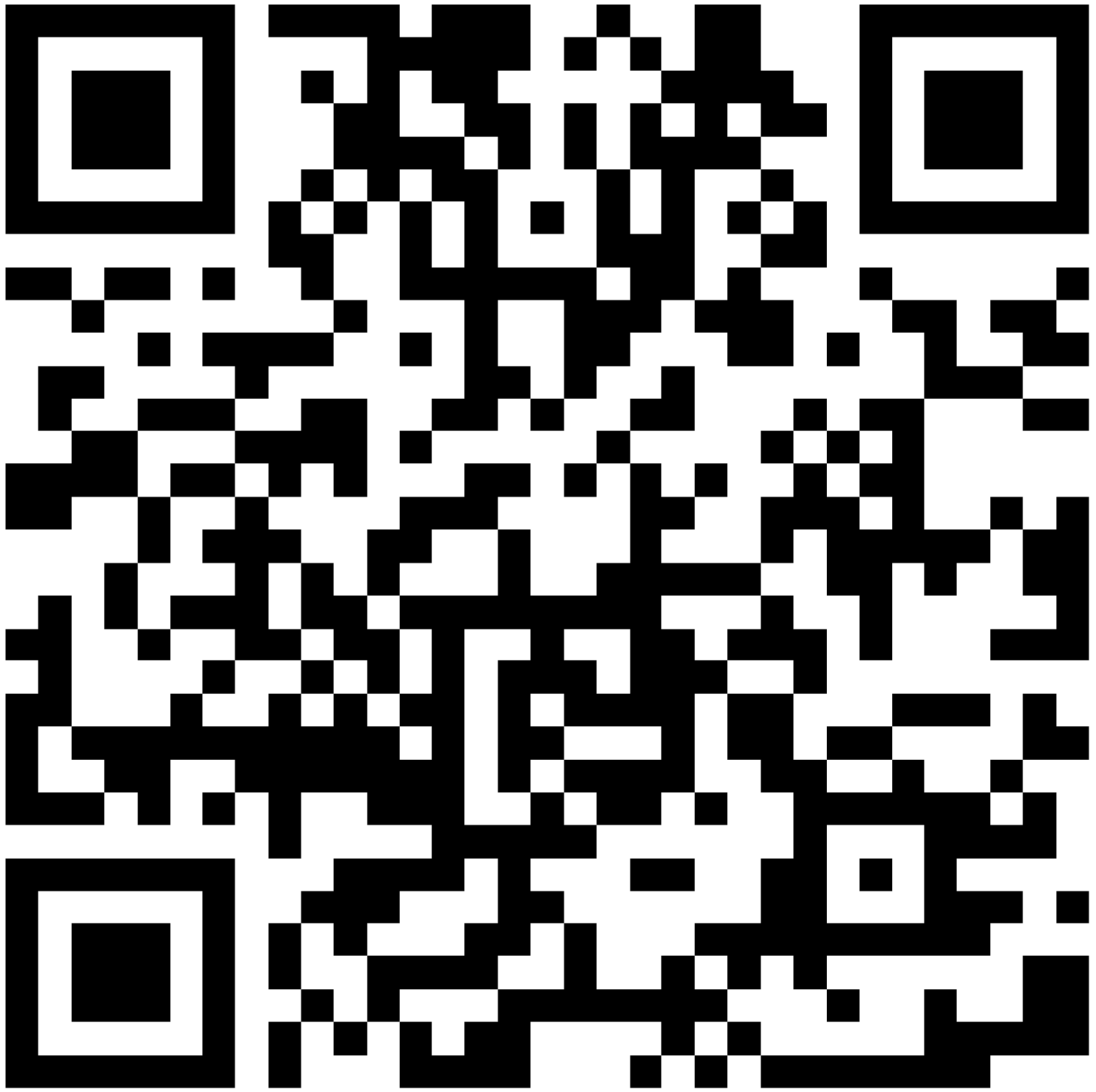
COMP4139 Machine Learning (MLE)

Deep Convolutional Neural Networks - Applications

Dr Shreyank N Gowda
Dr Direnc Pekaslan

School of Computer Science
University of Nottingham

Recap



Recap

- Pooling
- Number of parameters in a CNN
- Why convolution?
- Optimization approaches
- Exponential Averaging
- Momentum
- Adam

Learning Objectives

- Image Classification
- AlexNet
- VGG
- Residual Networks
- Data Augmentation
- Dropout
- Transfer Learning
- Image Segmentation
- Some demos!

Image Classification

Feature	ImageNet (ILSVRC)	PASCAL VOC 2012
Total Images	~1.2 million training, 50,000 validation, 150,000 testing	11,540 (training and validation)
Number of Categories	1,000	20
Complexity	High	Low

https://www.image-net.org/

University of Nottingham
UK | CHINA | MALAYSIA

ImageNet

Image Classification

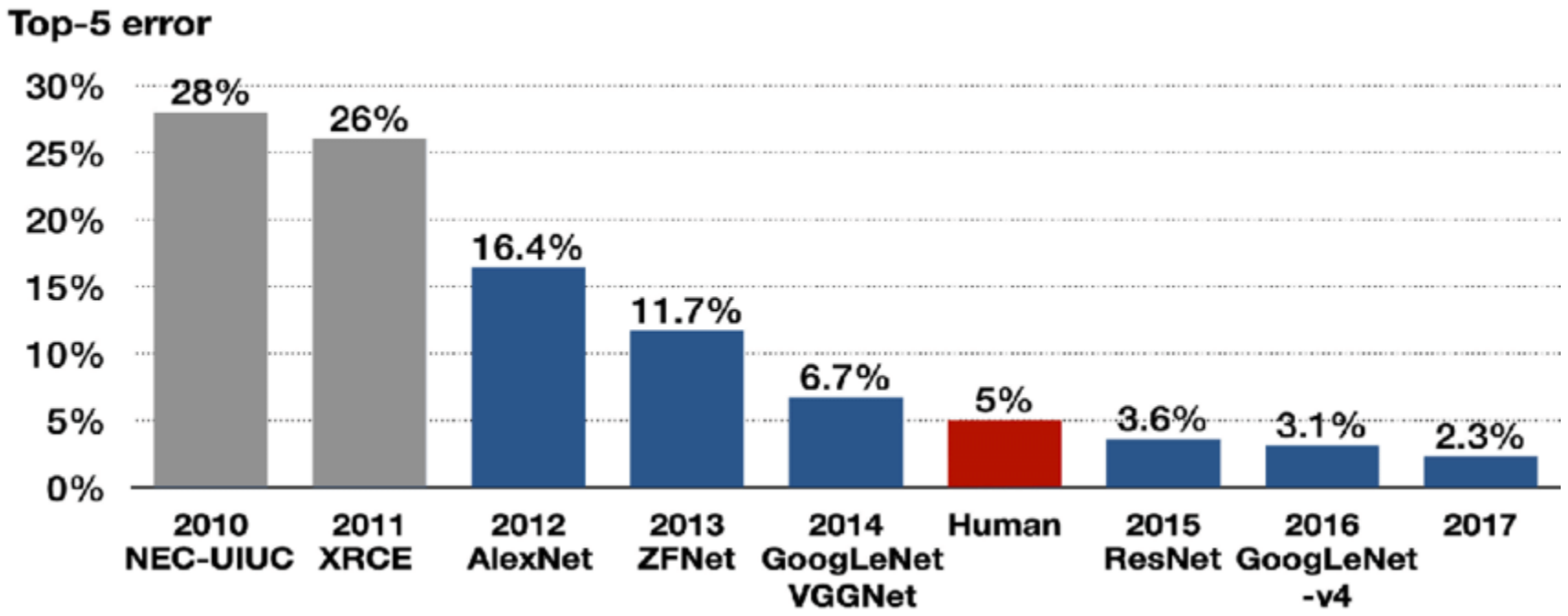
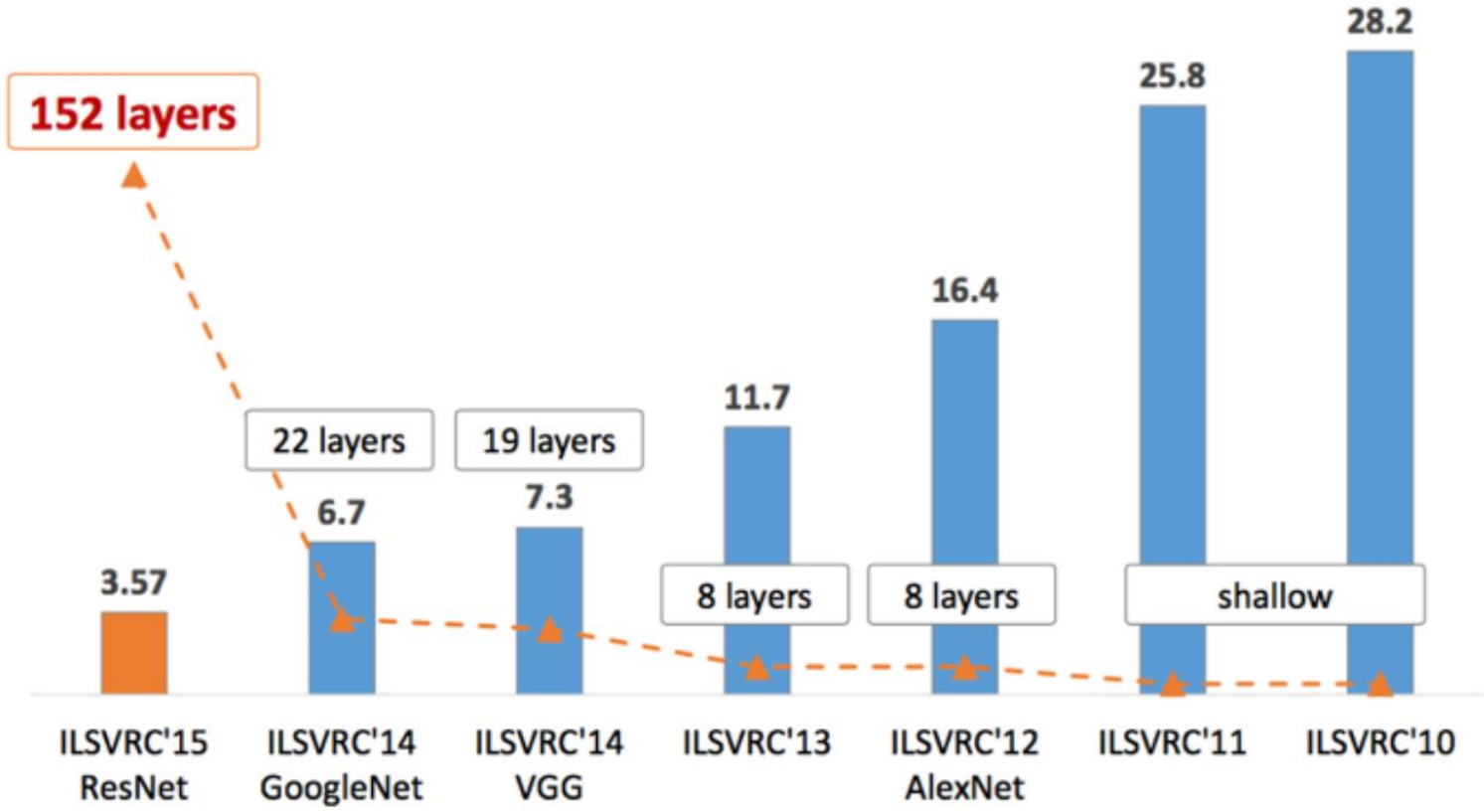


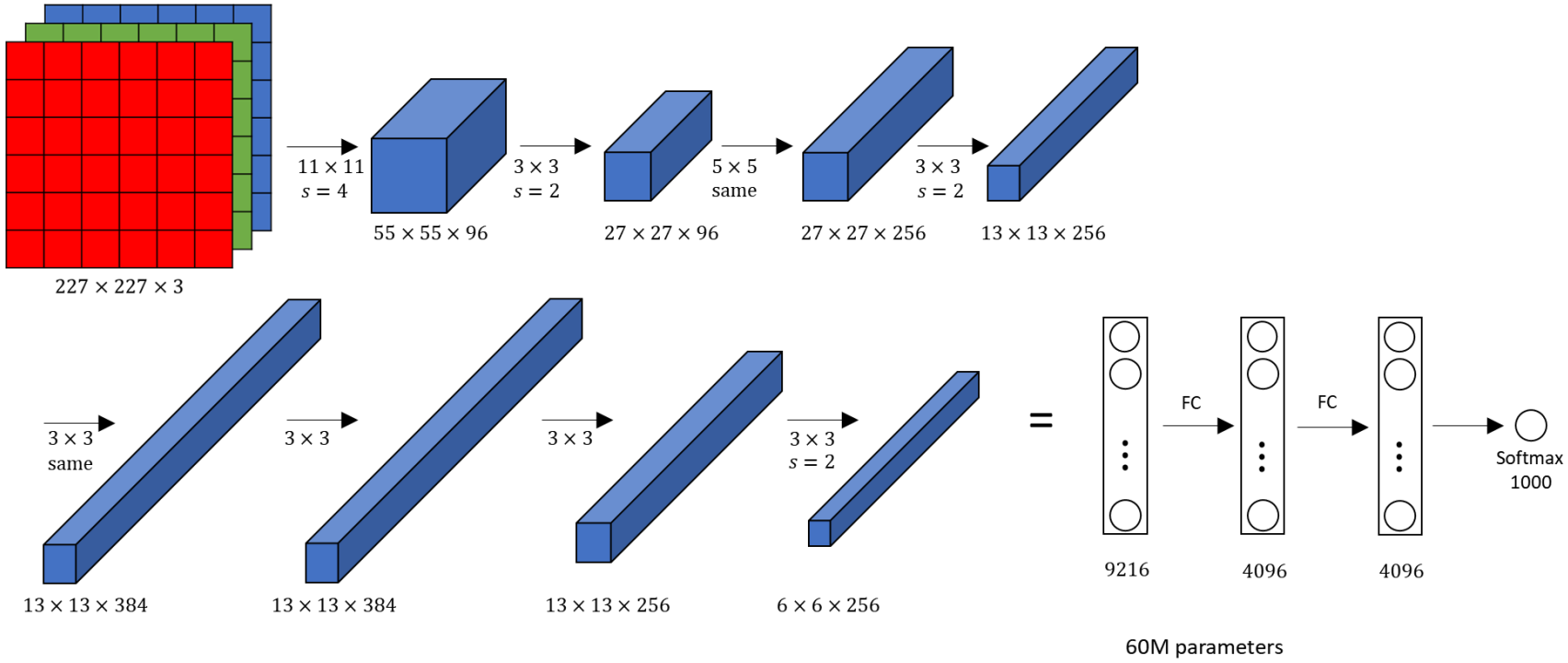
Image Classification



AlexNet

- AlexNet 2012 (>185K citations), University of Toronto:
  - Trained with stochastic gradient descent on 2 GPUs
  - 650,000 Neurons; 60 million parameters
  - Relu is used as activation function

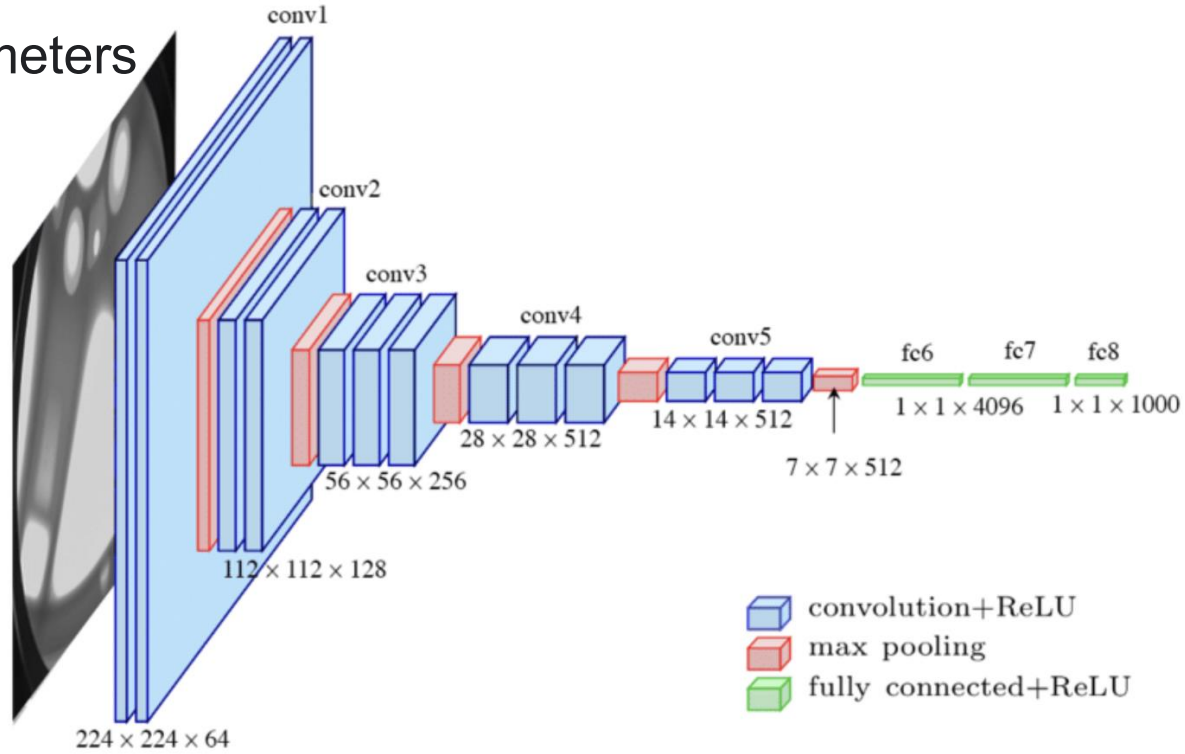
"Ilya thought we should do it, Alex made it work, and I got the Nobel Prize"



A. Krizhevsky et al., ImageNet Classification with Deep Convolutional Neural Networks, 2012.

VGG

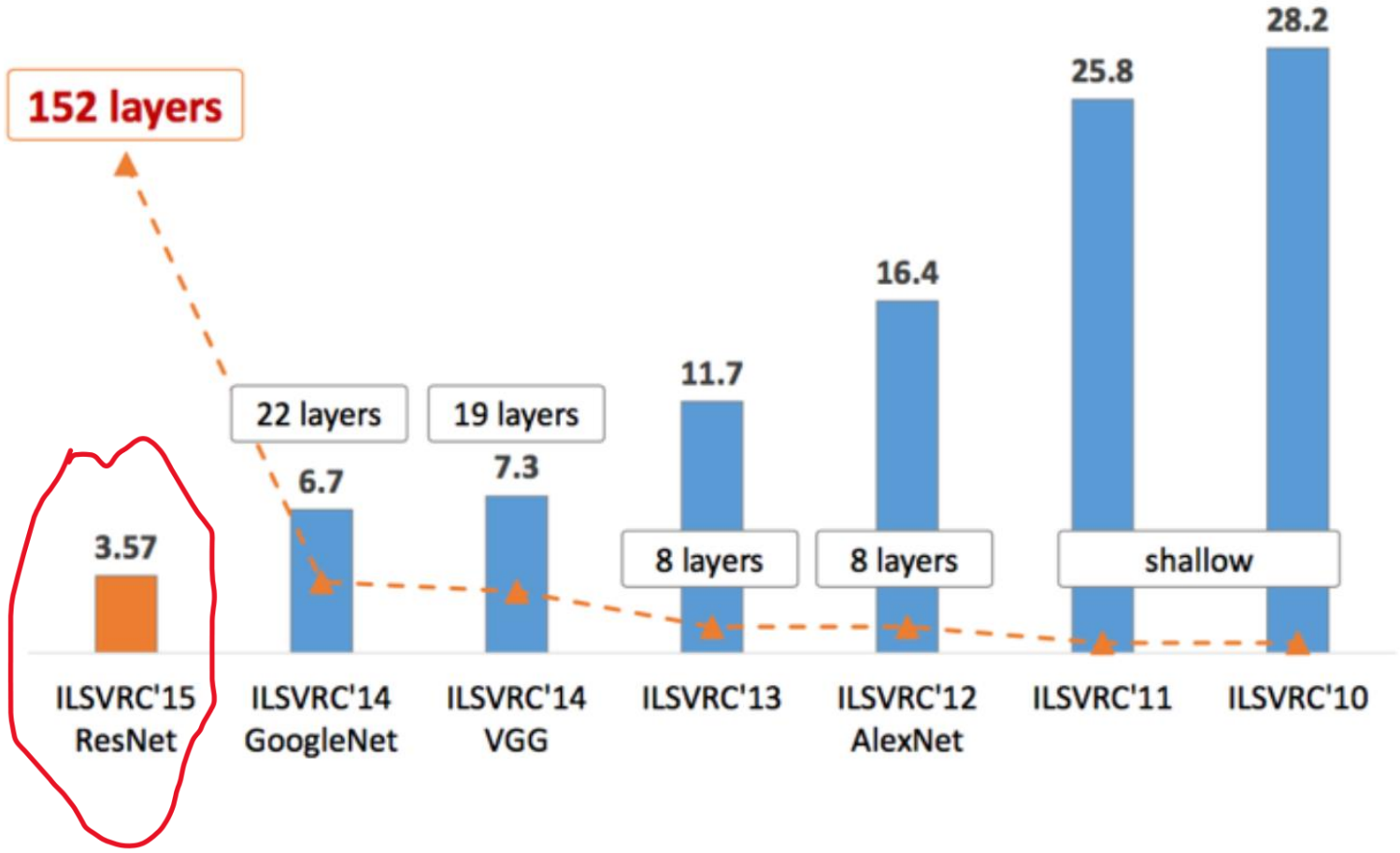
- VGG net 2014 (>150K citations), University of Oxford, Visual Geometry Group.
  - Different variants: VGG 16, VGG19
  - Only uses 3×3 kernels
  - VGG16 has a total of 138 million parameters



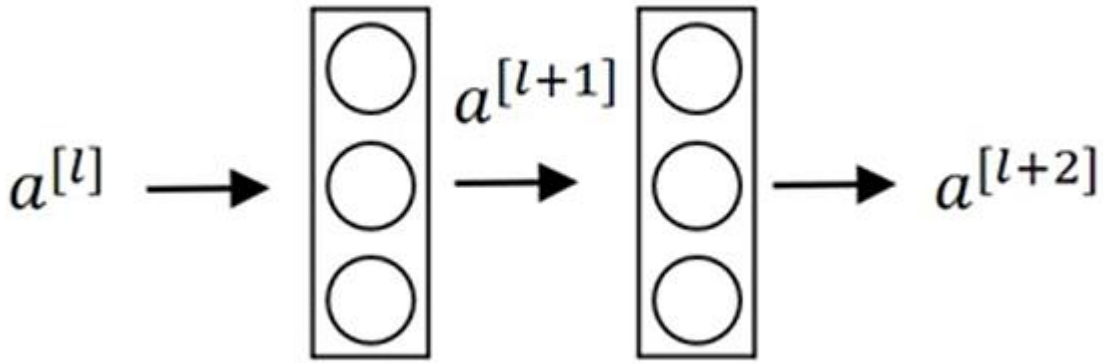
VGG 16

K. Simonyan et al. Very deep convolutional networks for large-scale visual recognition, 2014.

Residual Networks

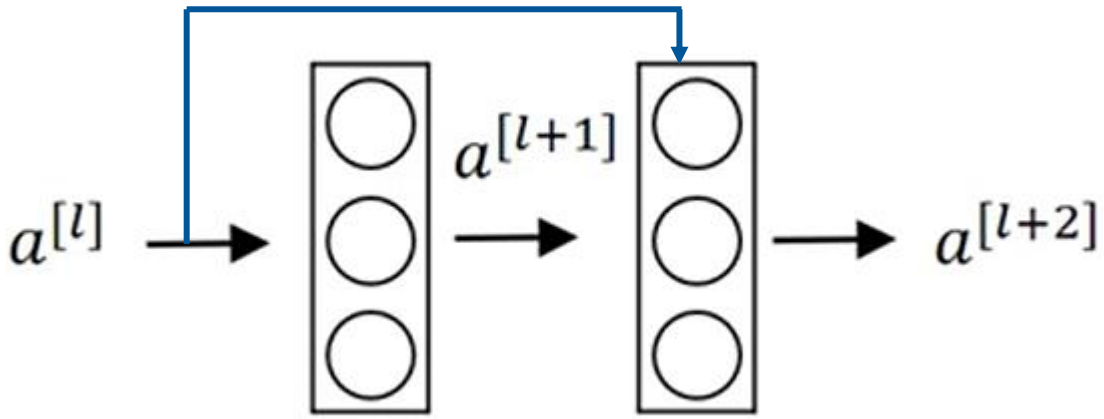


Residual Block



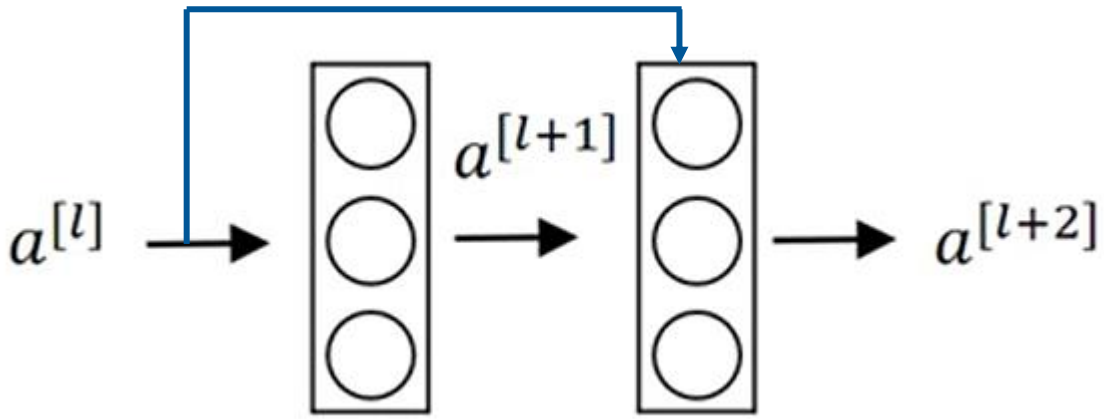
$$z^{[l+1]} = W^{[l+1]}\, a^{[l]} + b^{[l+1]} \qquad a^{[l+1]} = g(z^{[l+1]}) \qquad z^{[l+2]} = W^{[l+2]} a^{[l+1]} + b^{[l+2]} \qquad a^{[l+2]} = g(z^{[l+2]})$$

Residual Block



$$z^{[l+1]} = W^{[l+1]}\, a^{[l]} + b^{[l+1]} \qquad a^{[l+1]} = g(z^{[l+1]}) \qquad z^{[l+2]} = W^{[l+2]} a^{[l+1]} + b^{[l+2]} \qquad a^{[l+2]} = g(z^{[l+2]})$$

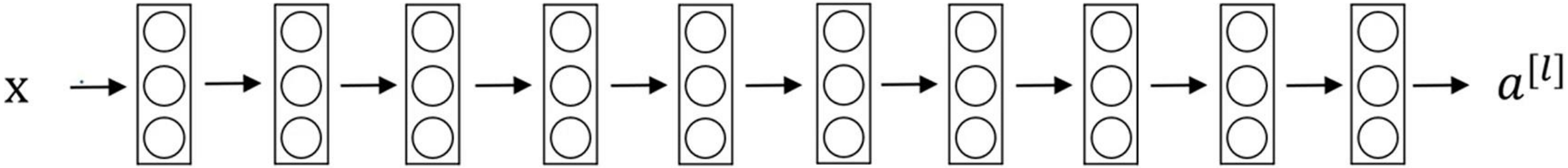
Residual Block



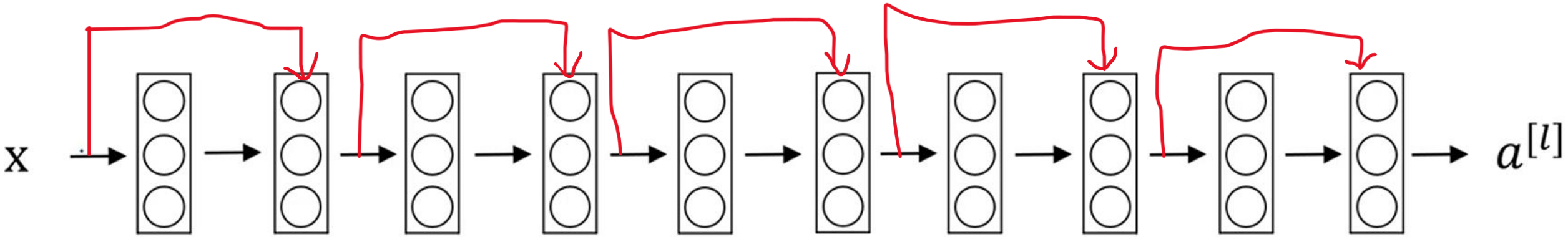
$$z^{[l+1]} = W^{[l+1]}\, a^{[l]} + b^{[l+1]} \qquad a^{[l+1]} = g(z^{[l+1]}) \qquad z^{[l+2]} = W^{[l+2]} a^{[l+1]} + b^{[l+2]} \qquad a^{[l+2]} = g(z^{[l+2]})$$

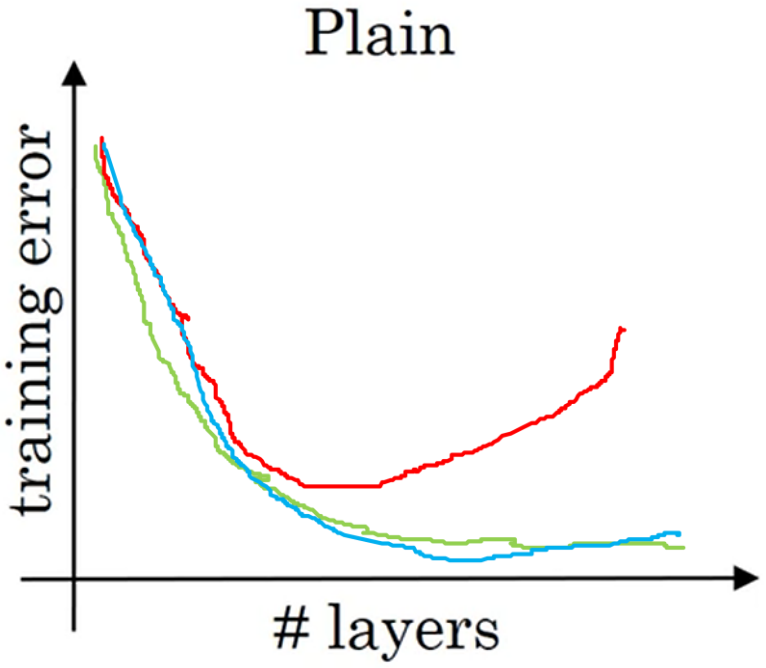
$$a^{[l+2]} = g\left(z^{[l+2]} + a^{[l]}\right)$$

Residual Network



Residual Network

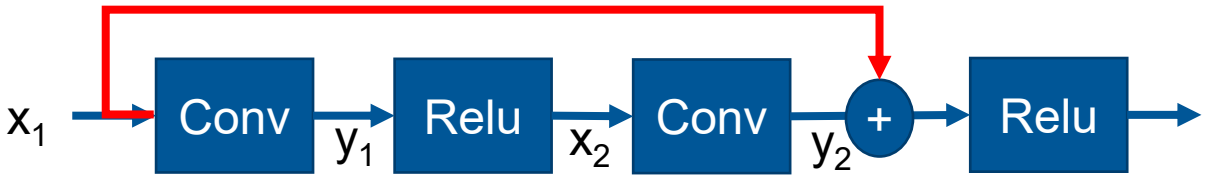




Theory
Reality
Residual Nets

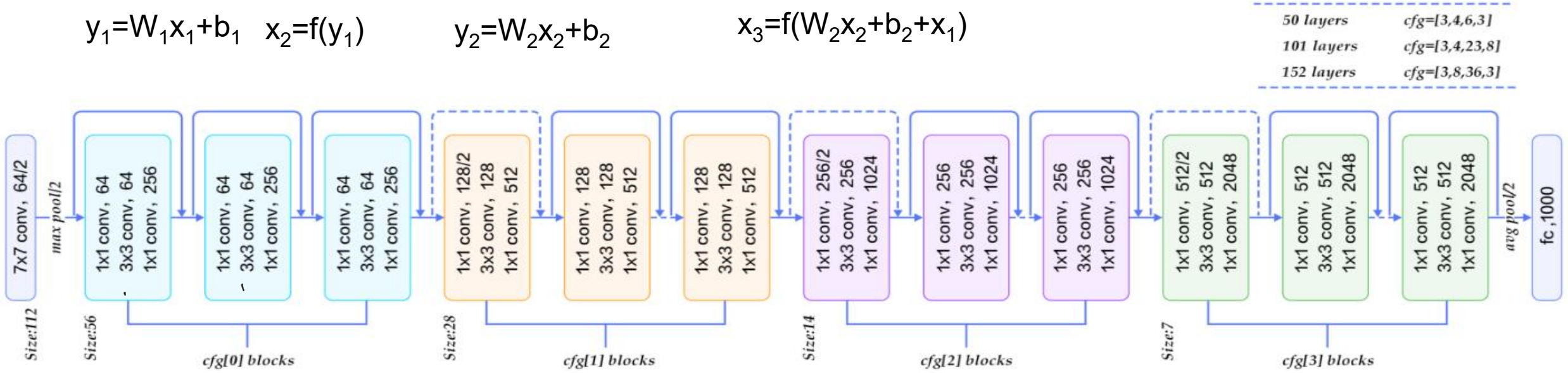
Residual Network (~300K citations)

- The deeper the better for learning but harder to learn.
- Resnet18, 50, 101, Resnet 101 has 44.5 million parameters



$y_1=W_1x_1+b_1$ $x_2=f(y_1)$ $y_2=W_2x_2+b_2$ $x_3=f(W_2x_2+b_2+x_1)$

If $W_2=0$ and $b_2=0$ and assume Relu is used, we have $x_3= x_1$ (identity is easy to be learned)



K. He et al., Deep Residual Learning for Image Recognition, 2014.

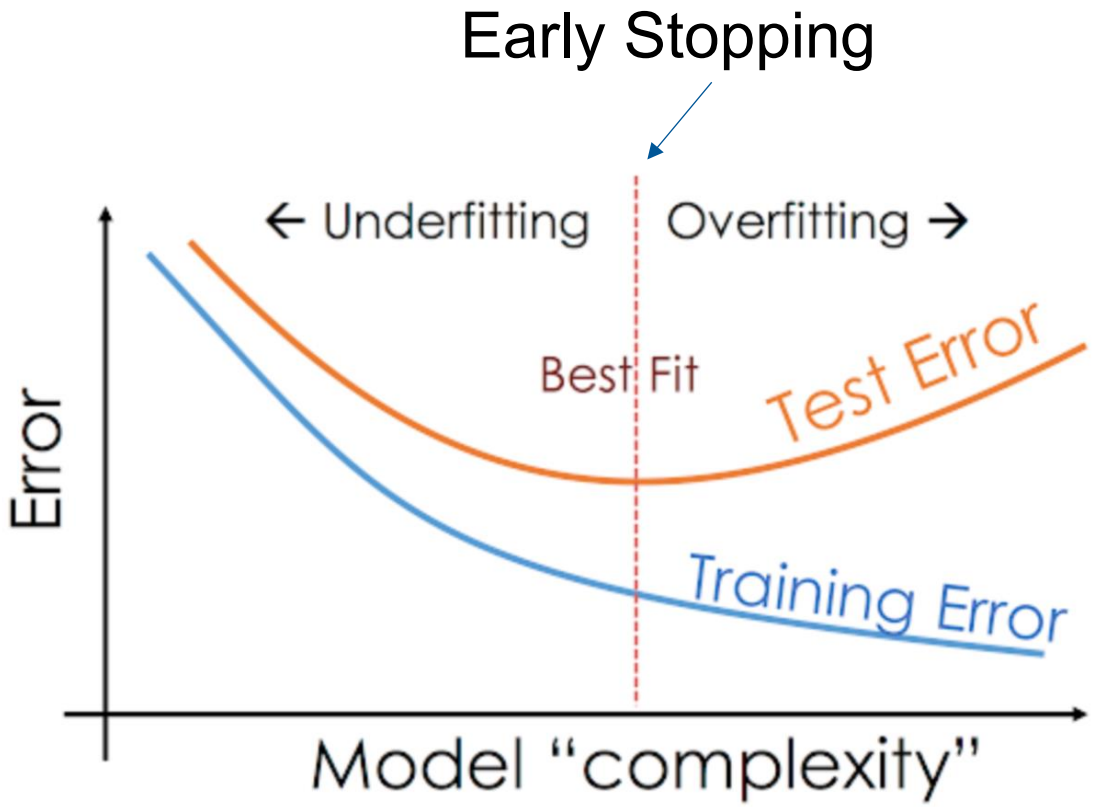
Overfitting is a big problem in deep learning

Problem:

- Alexnet has 60 million parameters
- VGG16 has a total of 138 million parameters
- Resnet101 has 44.5 million parameters

Method to reduce overfitting:

- Regularization
- Data Augmentation
- Drop out
- Transfer learning



Regularization

L1 Regularization

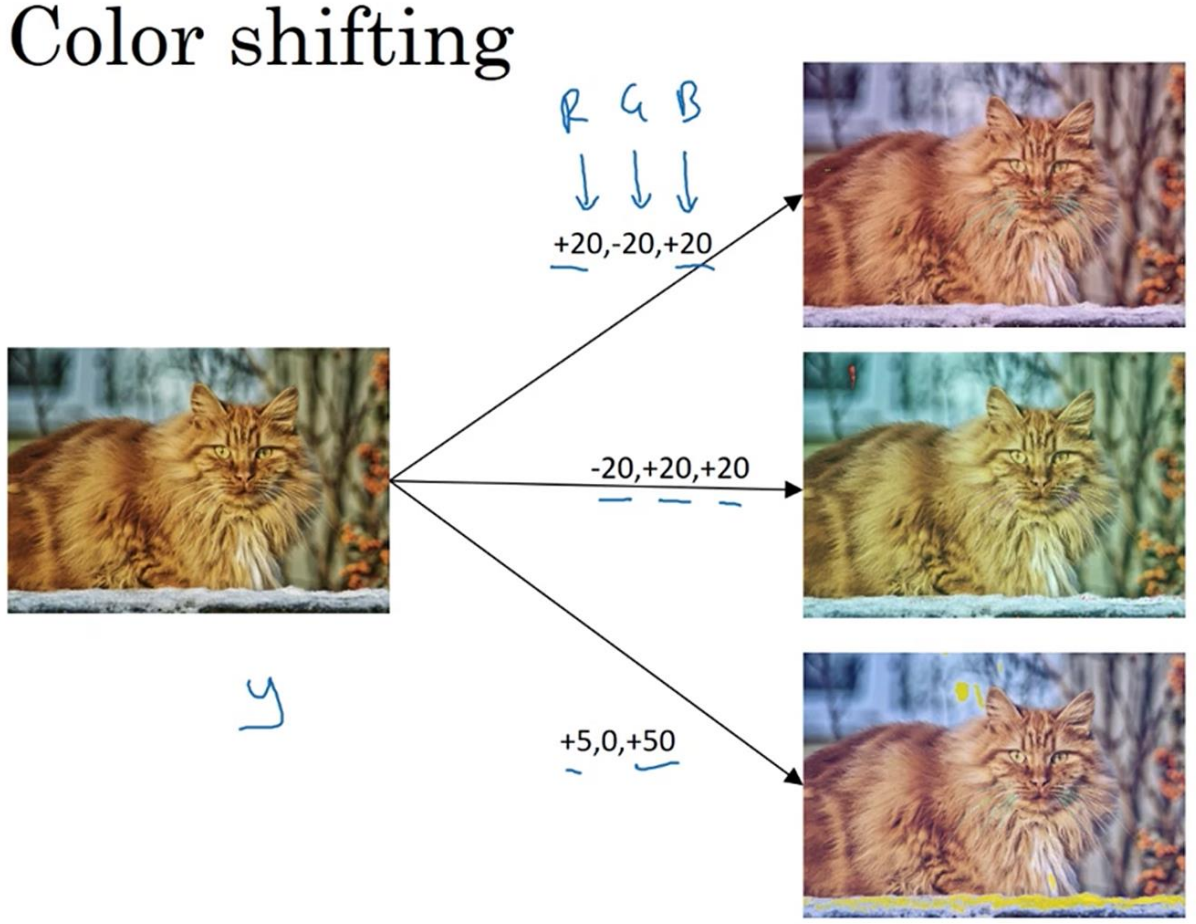
$$\text{Cost} = \sum_{i=0}^{N} (y_i - \sum_{j=0}^{M} x_{ij} W_j)^2 + \lambda \sum_{j=0}^{M} |W_j|$$

L2 Regularization

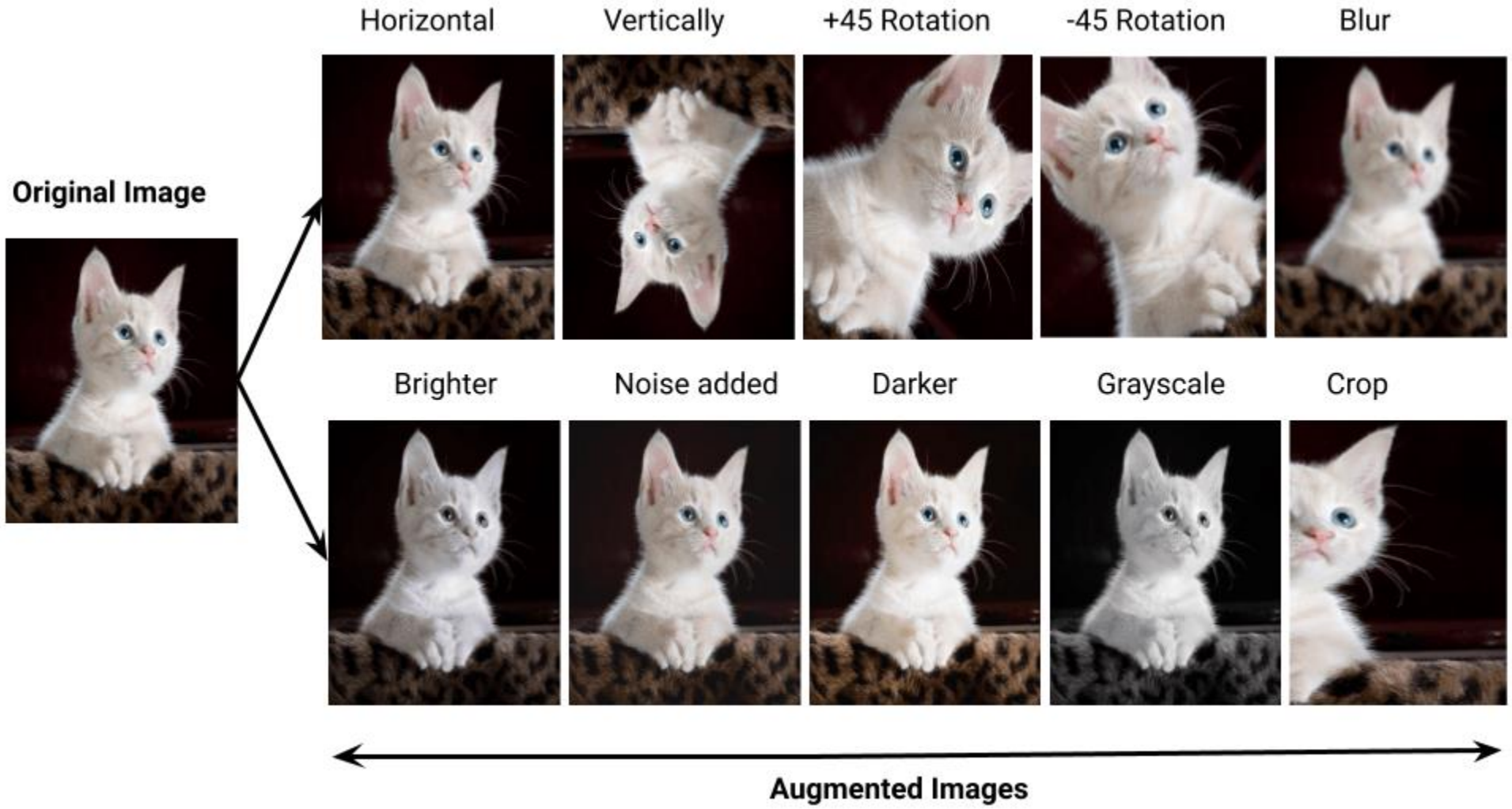
$$\text{Cost} = \underbrace{\sum_{i=0}^{N} (y_i - \sum_{j=0}^{M} x_{ij} W_j)^2}_{\text{Loss function}} + \underbrace{\lambda \sum_{j=0}^{M} W_j^2}_{\text{Regularization Term}}$$

Data Augmentation

- Data augmentation increases the number of training samples by varying the original image's geometry and appearance.
- Data augmentation is normally performed during training not beforehand.



Data Augmentation

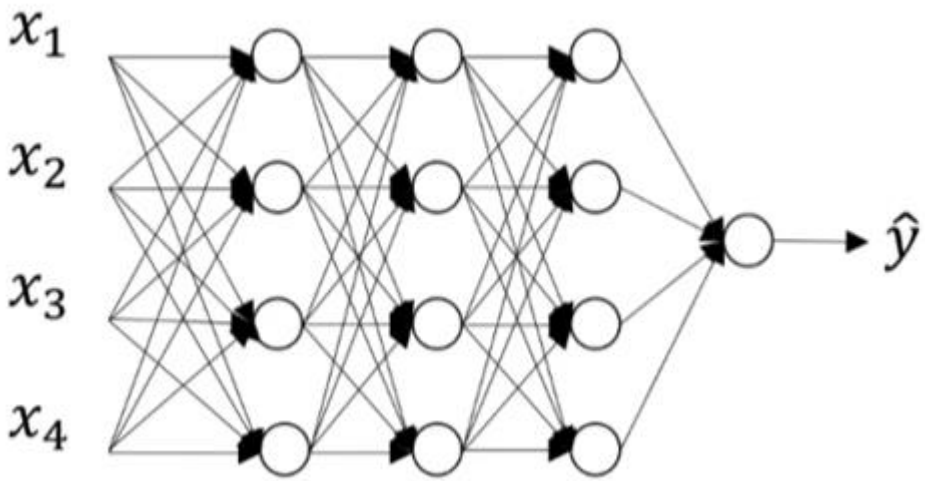


Data Augmentation

	ResNet-50	Mixup	Cutout	CutMix
Image				
Label	Dog 1.0	Dog 0.5 Cat 0.5	Dog 1.0	Dog 0.6 Cat 0.4
ImageNet Cls (%)	76.3 (+0.0)	77.4 (+1.1)	77.1 (+0.8)	**78.4** **(+2.1)**
ImageNet Loc (%)	46.3 (+0.0)	45.8 (-0.5)	46.7 (+0.4)	**47.3** **(+1.0)**
Pascal VOC Det (mAP)	75.6 (+0.0)	73.9 (-1.7)	75.1 (-0.5)	**76.7** **(+1.1)**

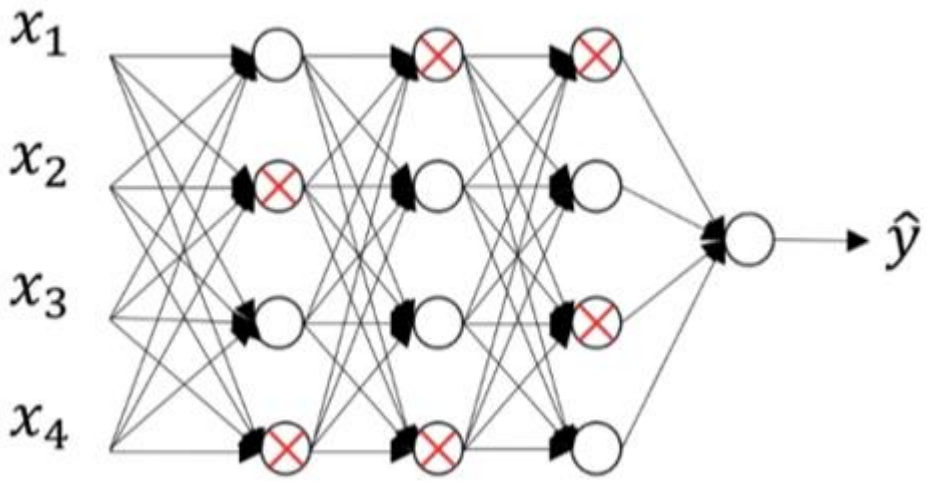
Dropout

- Randomly disable certain neurons during training.
- Normally in fully connected layer, e.g. 20% dropout rate



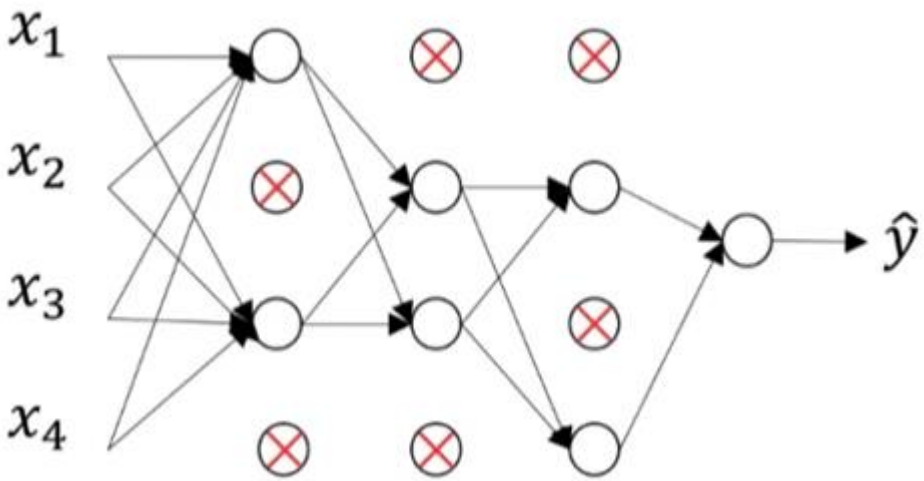
Dropout

- Randomly disable certain neurons during training.
- Normally in fully connected layer, e.g. 20% dropout rate



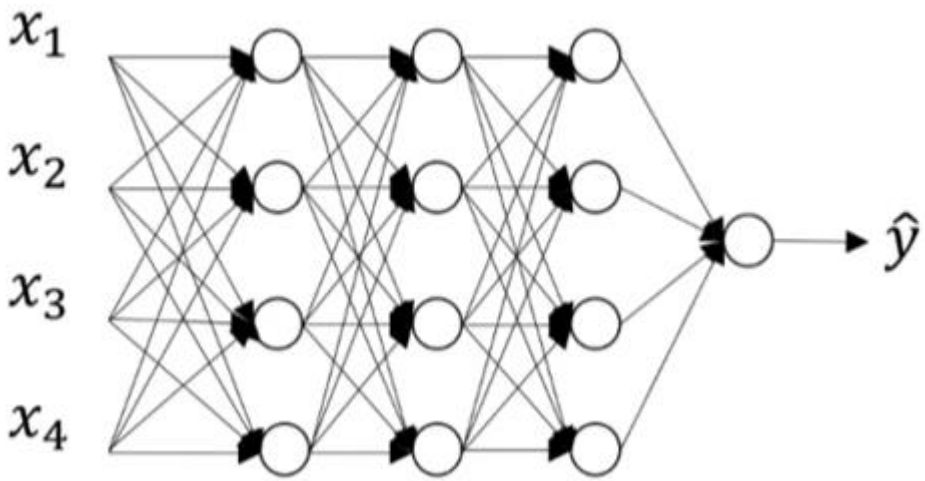
Dropout

- Randomly disable certain neurons during training.
- Normally in fully connected layer, e.g. 20% dropout rate

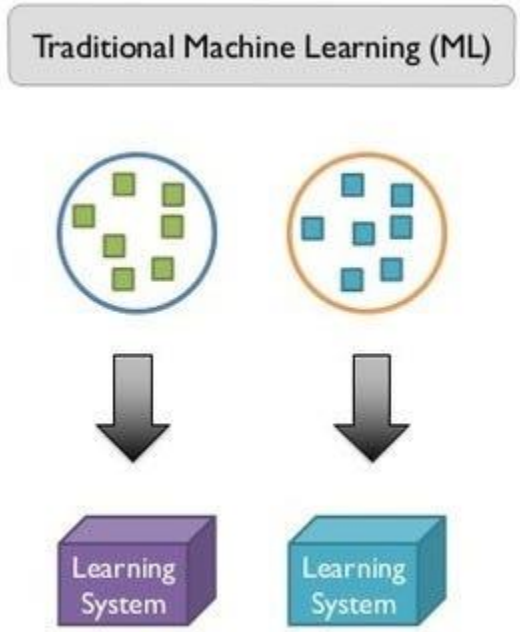


Dropout

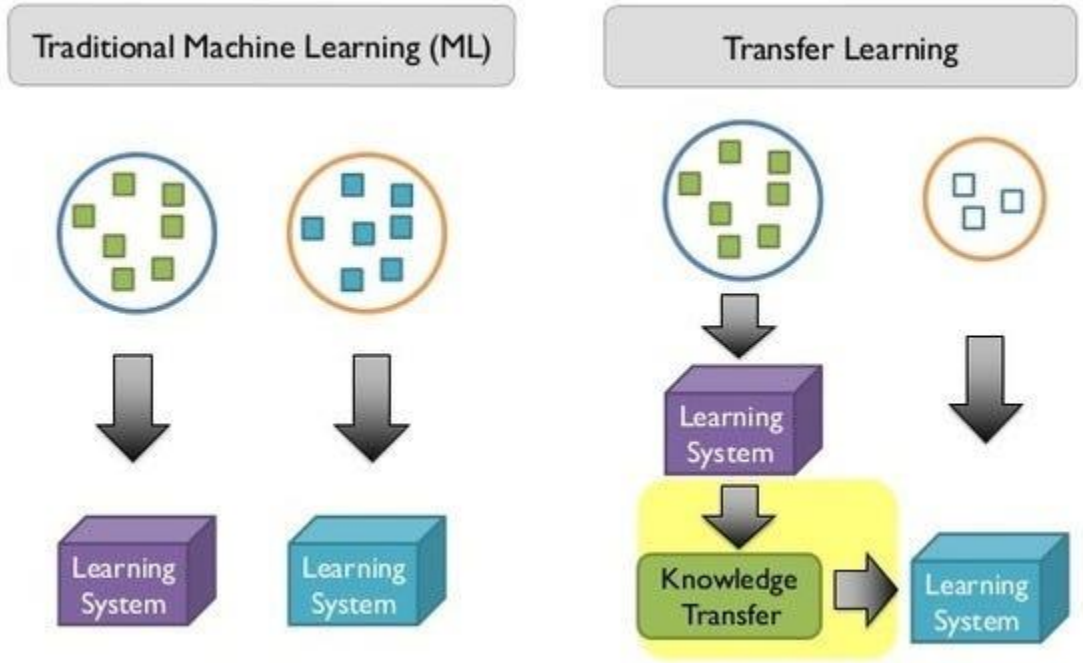
- At test time, full network is used.



Transfer Learning

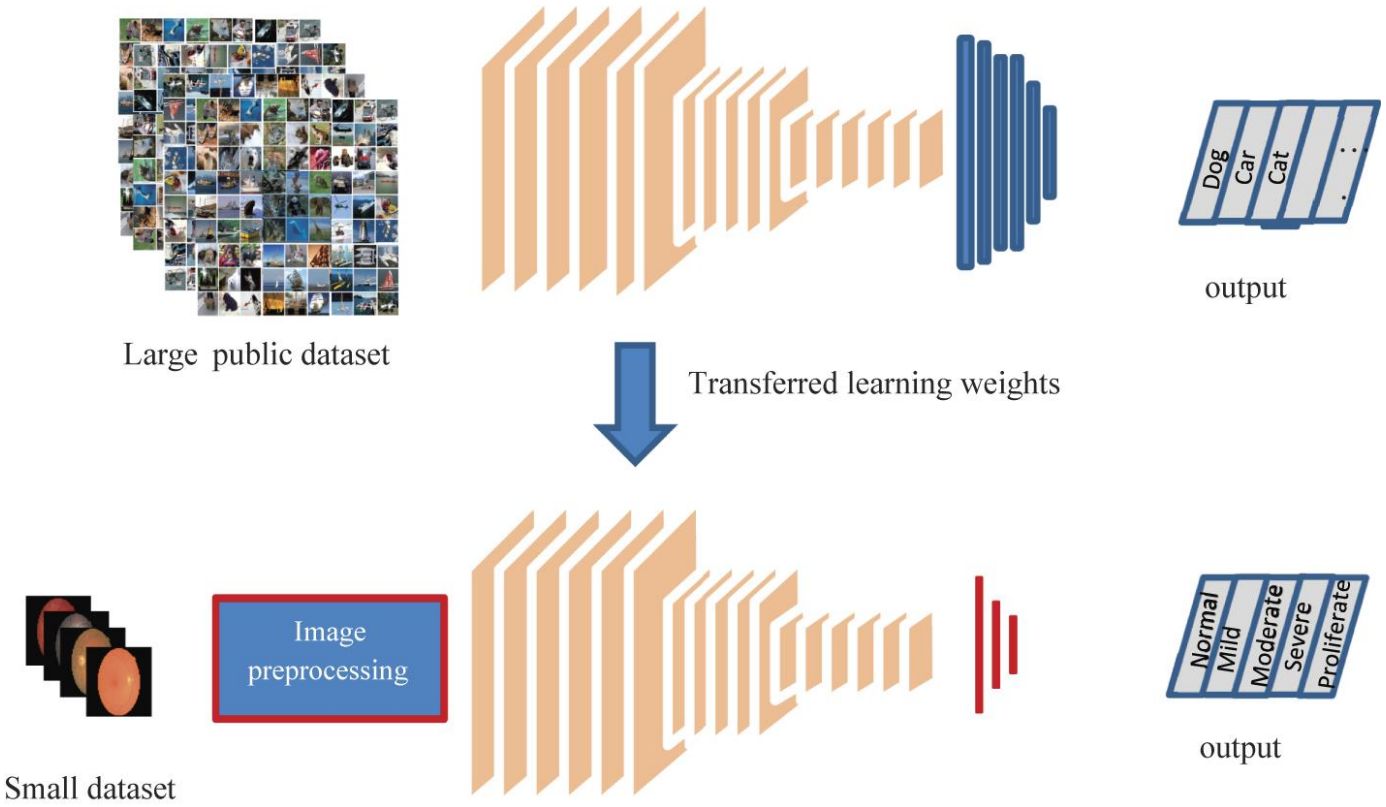


Transfer Learning



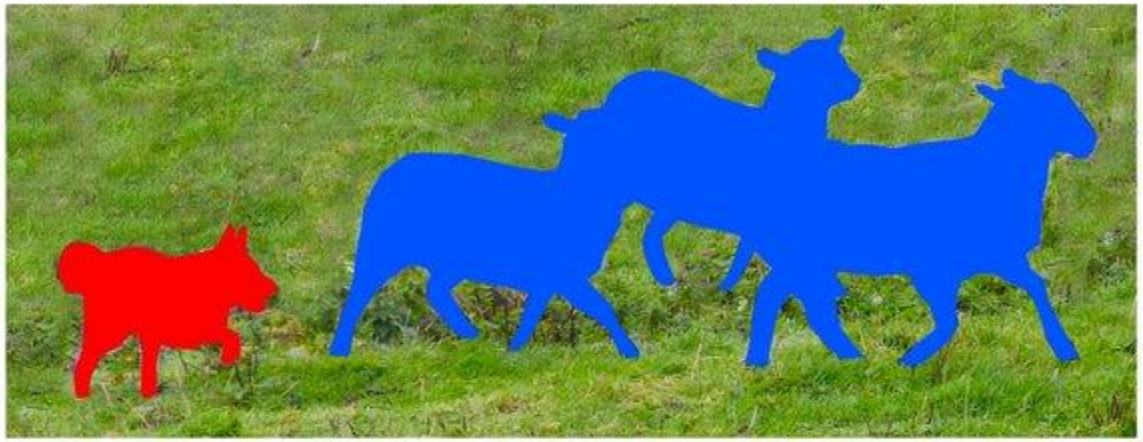
Transfer Learning

- Train the DCNN using a large dataset (e.g. imageNet).
- Then freeze the parameters of the first few layers, only retrain (fine tune) the high-level feature layers or fully connected layers for a new application with small number of training samples.
- Any restrictions using transfer learning?

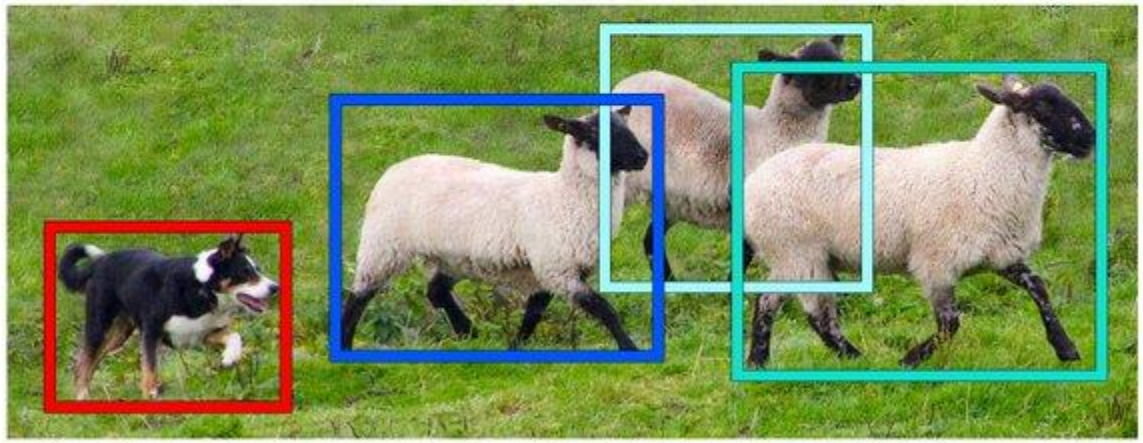


Common tasks

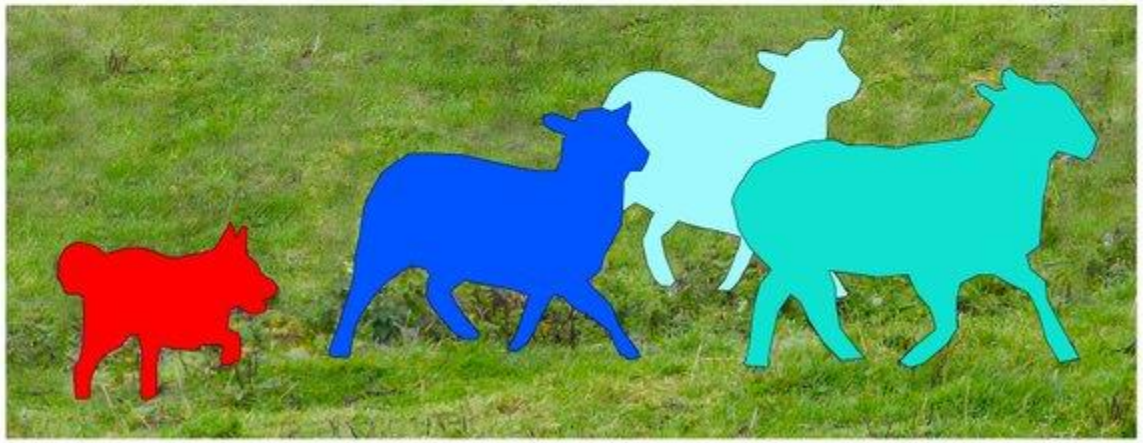
Image Recognition



Semantic Segmentation

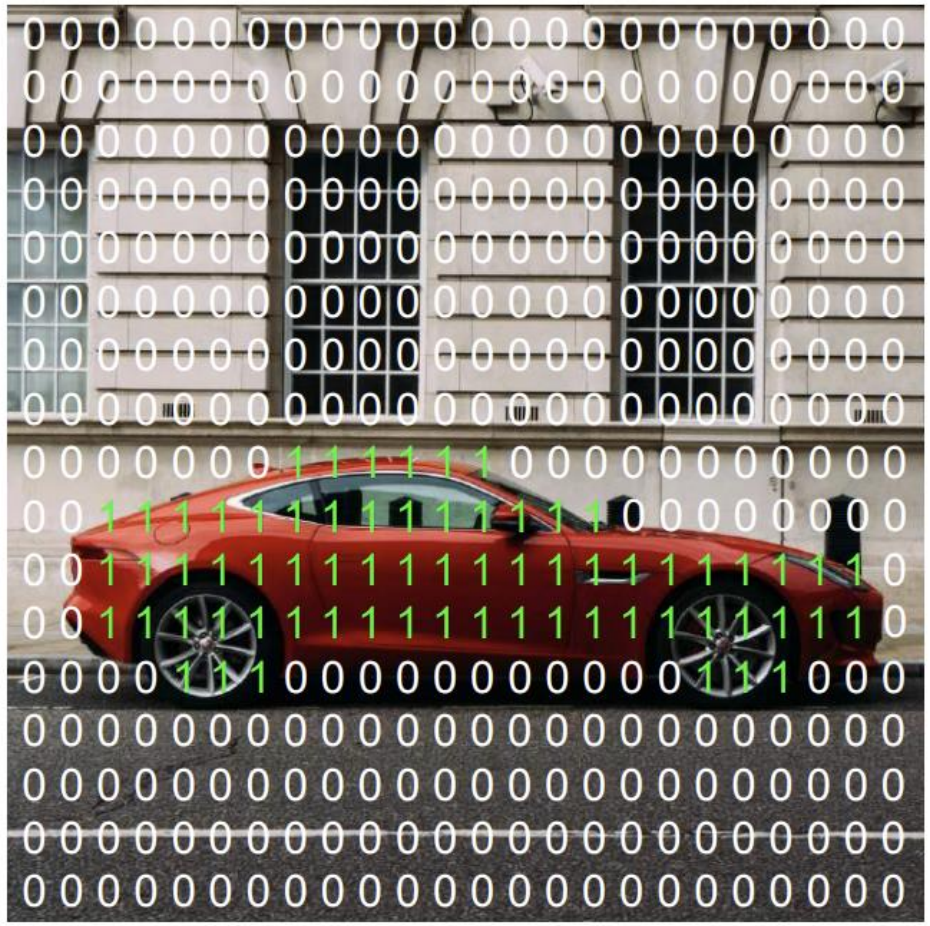


Object Detection



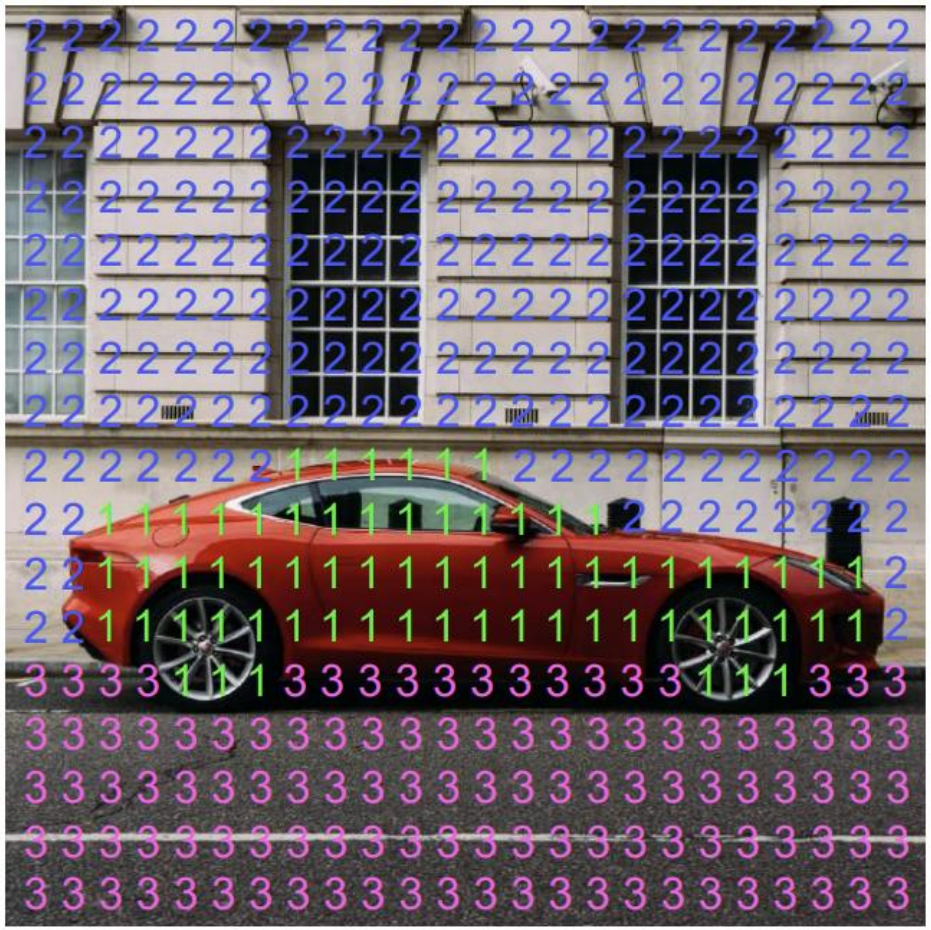
Instance Segmentation

Semantic Segmentation

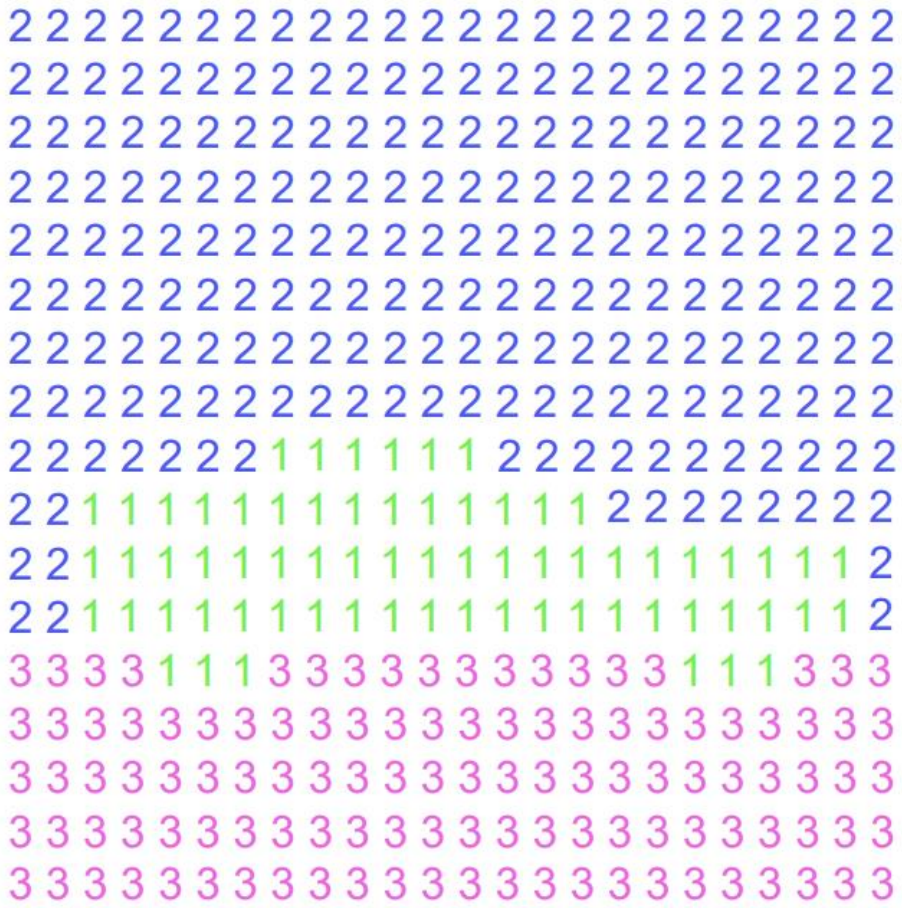


1. Car
0. Not Car

Semantic Segmentation

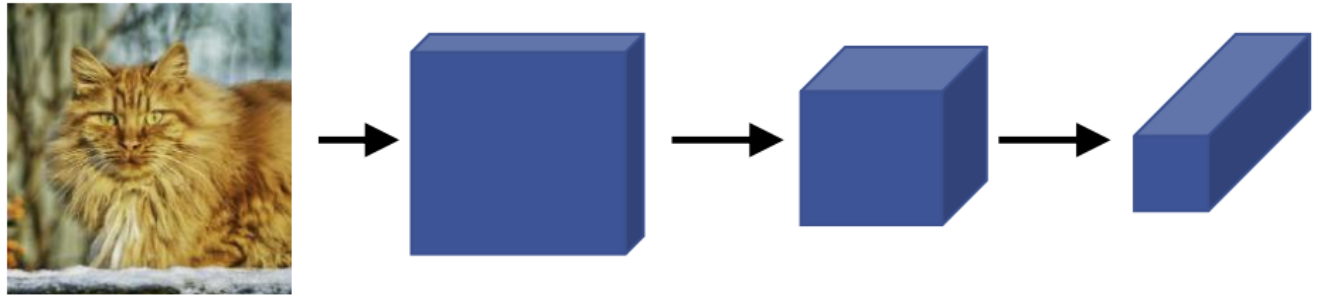


1. Car
2. Building
3. Road



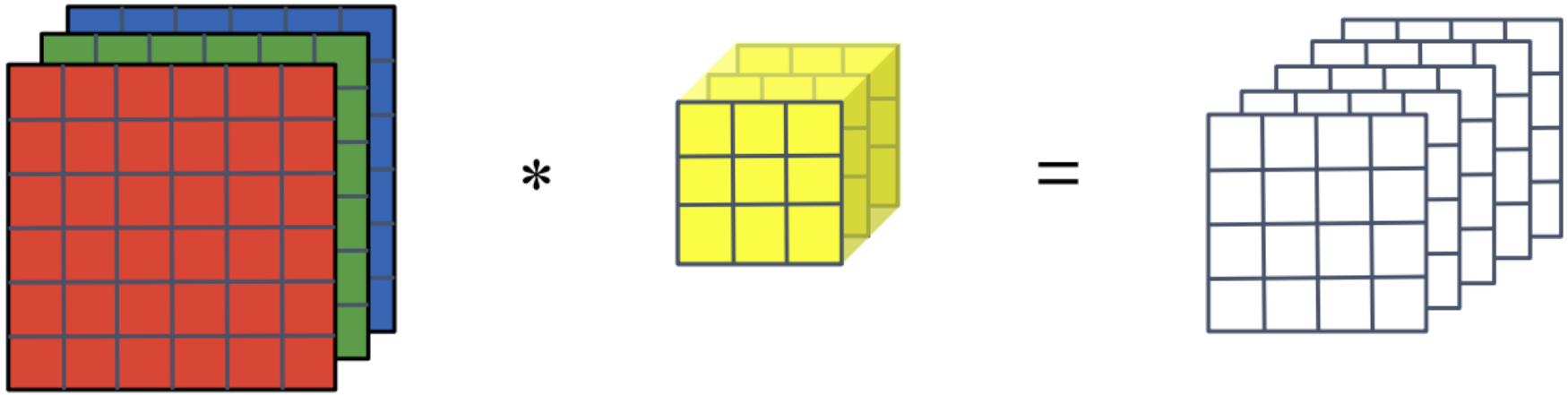
Segmentation Map

Semantic Segmentation

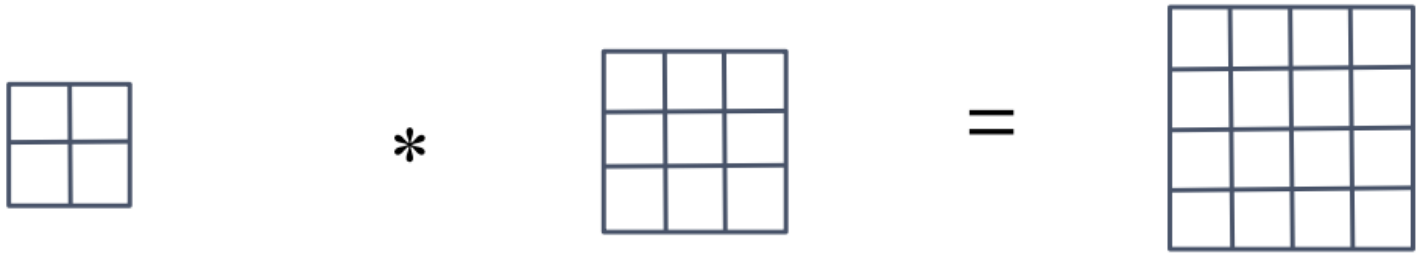


Transpose Convolution

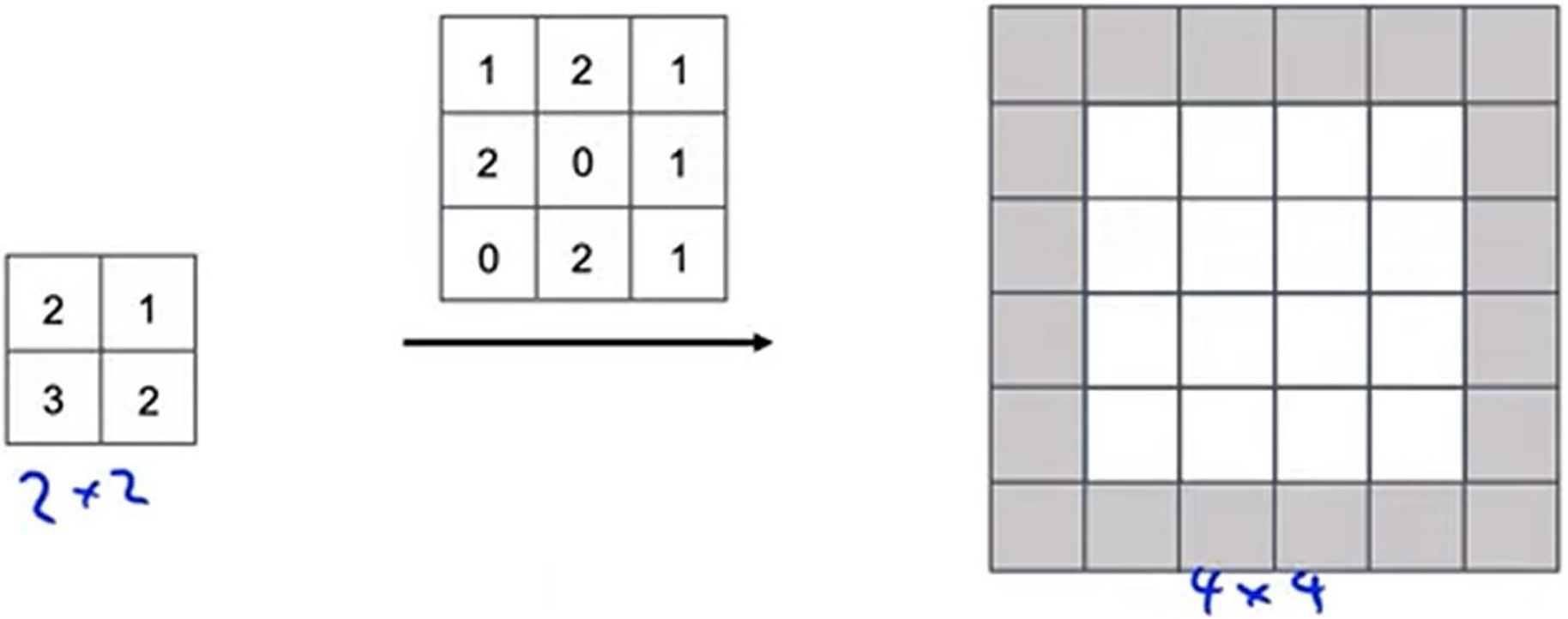
Normal Convolution



Transpose Convolution



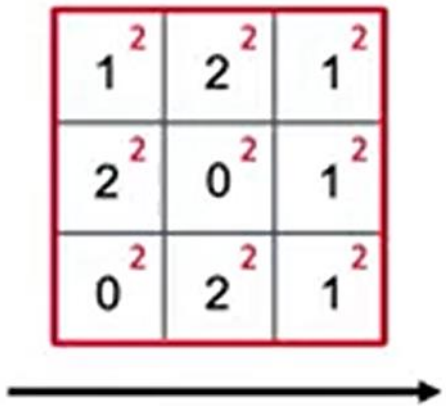
Transpose Convolution

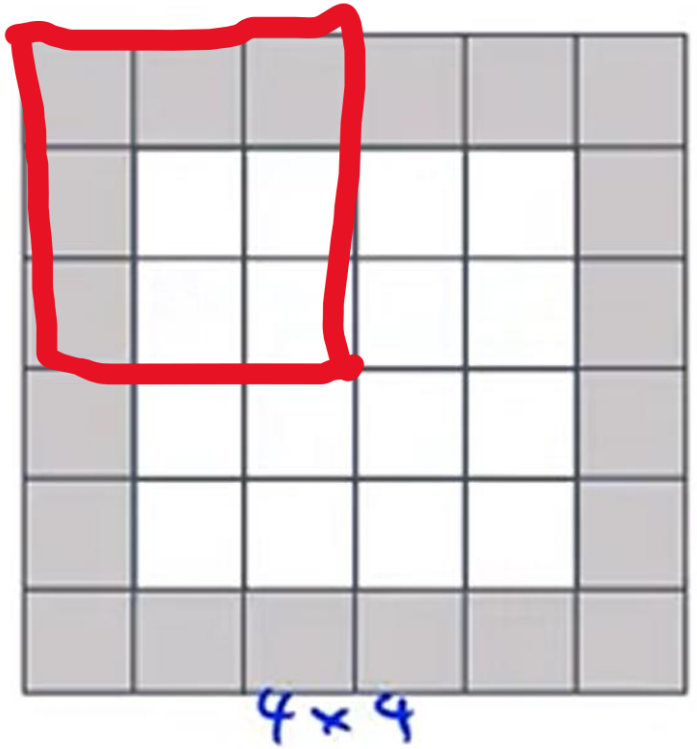


filter $f \times f = 3 \times 3$ padding $p = 1$ stride $s = 2$

Transpose Convolution

2×2



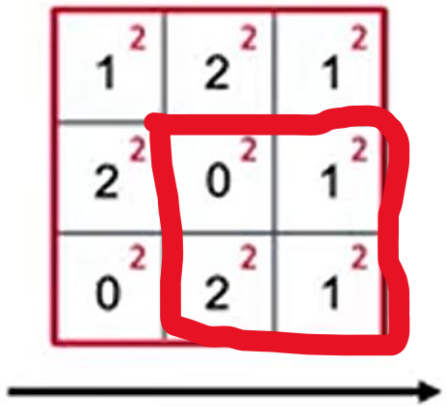


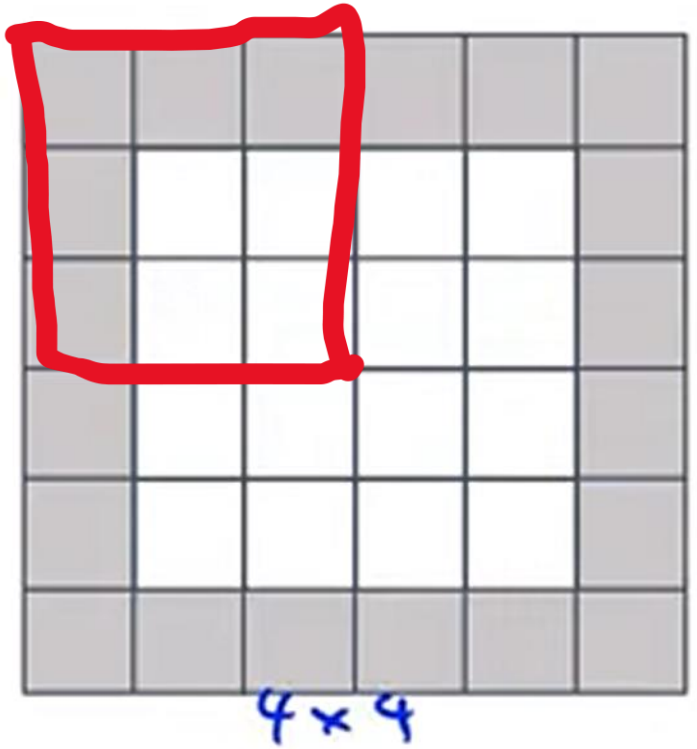
4×4

filter f×f = 3×3 padding p=1 stride s=2

Transpose Convolution

2×2



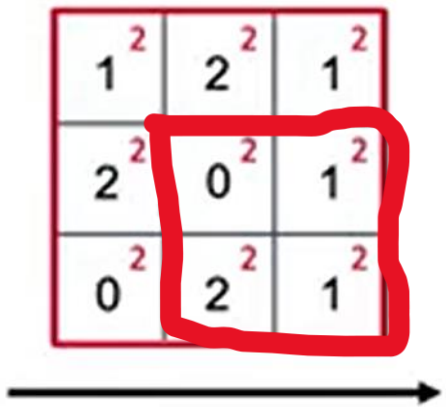


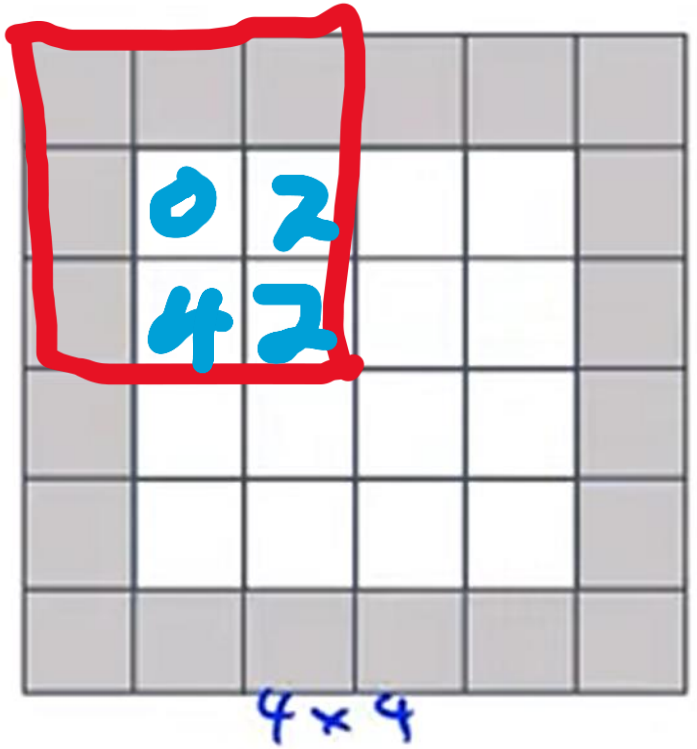
4×4

filter f×f = 3×3 padding p=1 stride s=2

Transpose Convolution

2×2



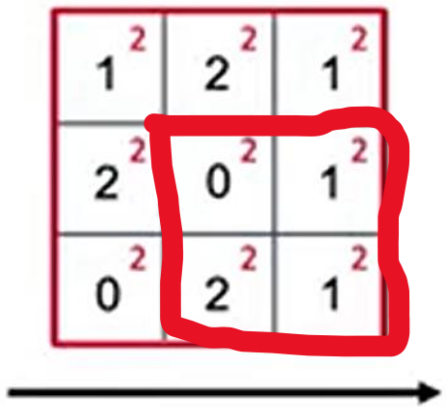


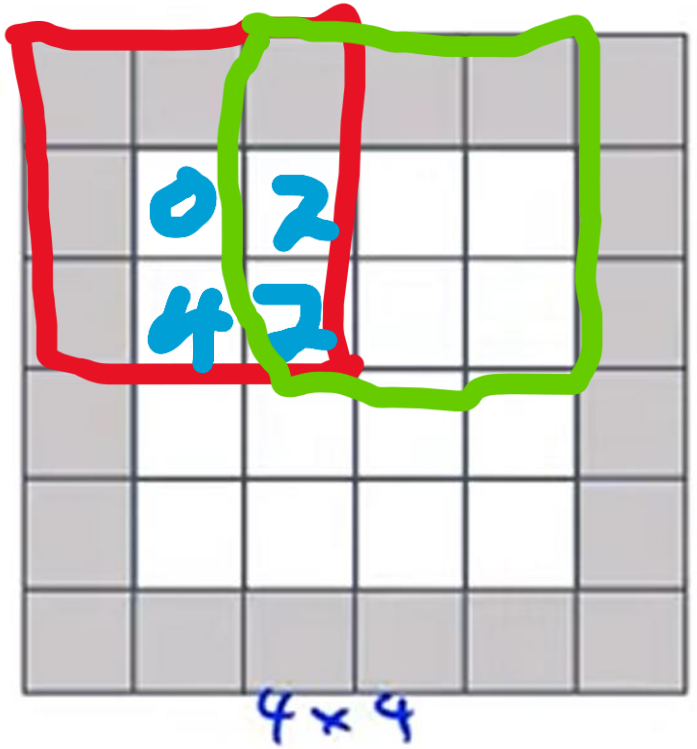
4×4

filter f×f = 3×3 padding p=1 stride s=2

Transpose Convolution

2×2



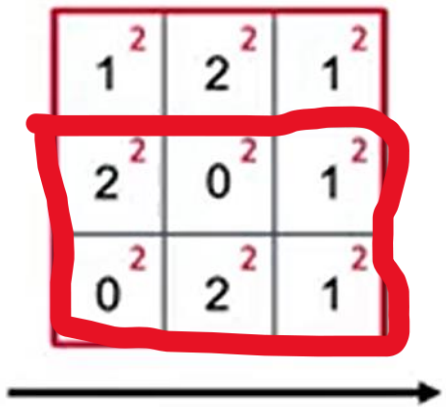


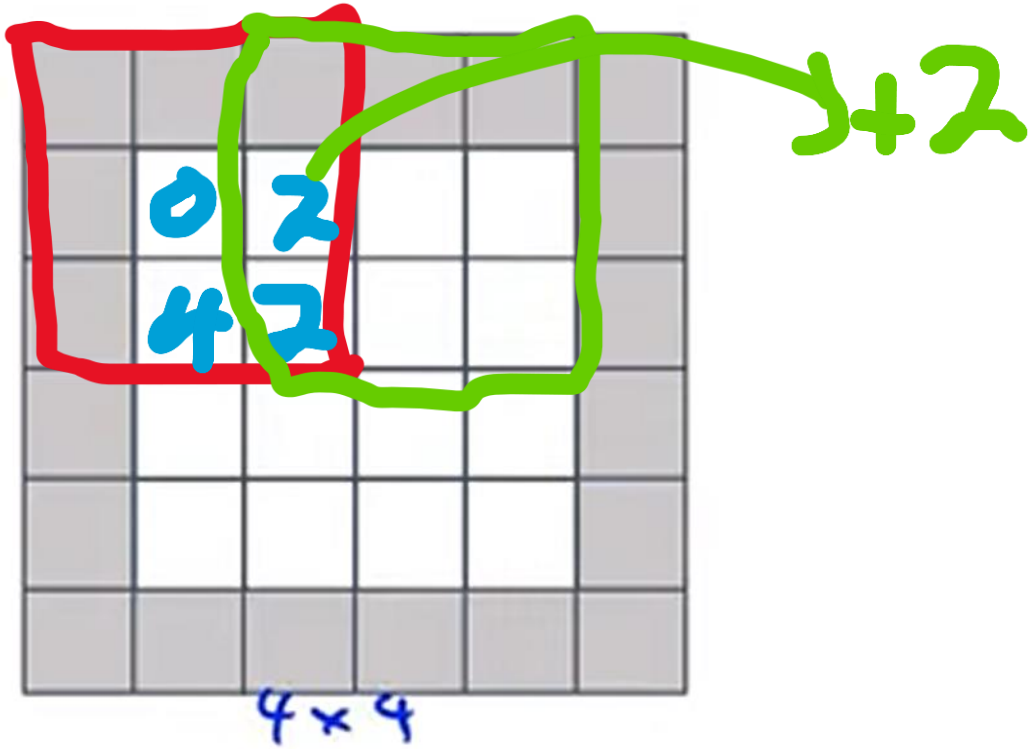
4×4

filter f×f = 3×3 padding p=1 stride s=2

Transpose Convolution

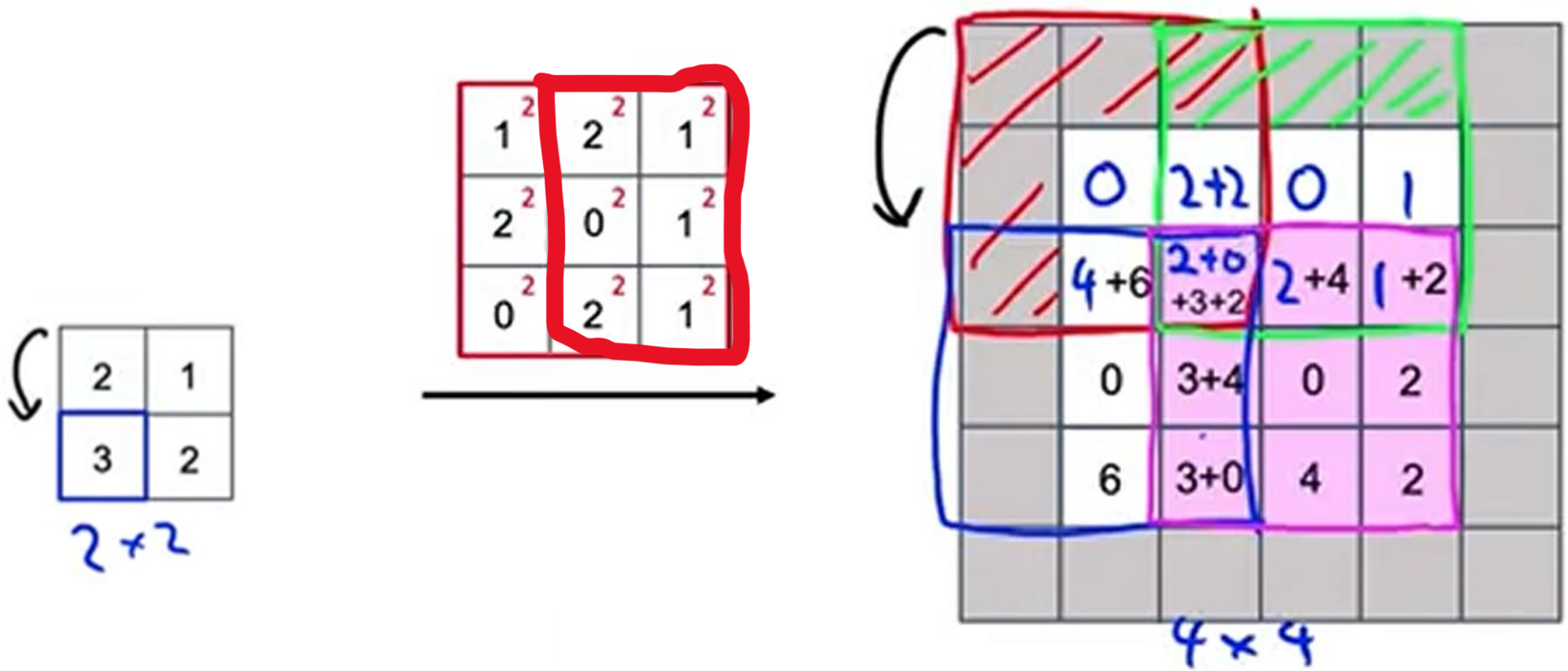
2×2





filter $f \times f = 3 \times 3$ padding $p = 1$ stride $s = 2$

Transpose Convolution



filter $f \times f = 3 \times 3$ padding $p = 1$ stride $s = 2$

Semantic Segmentation

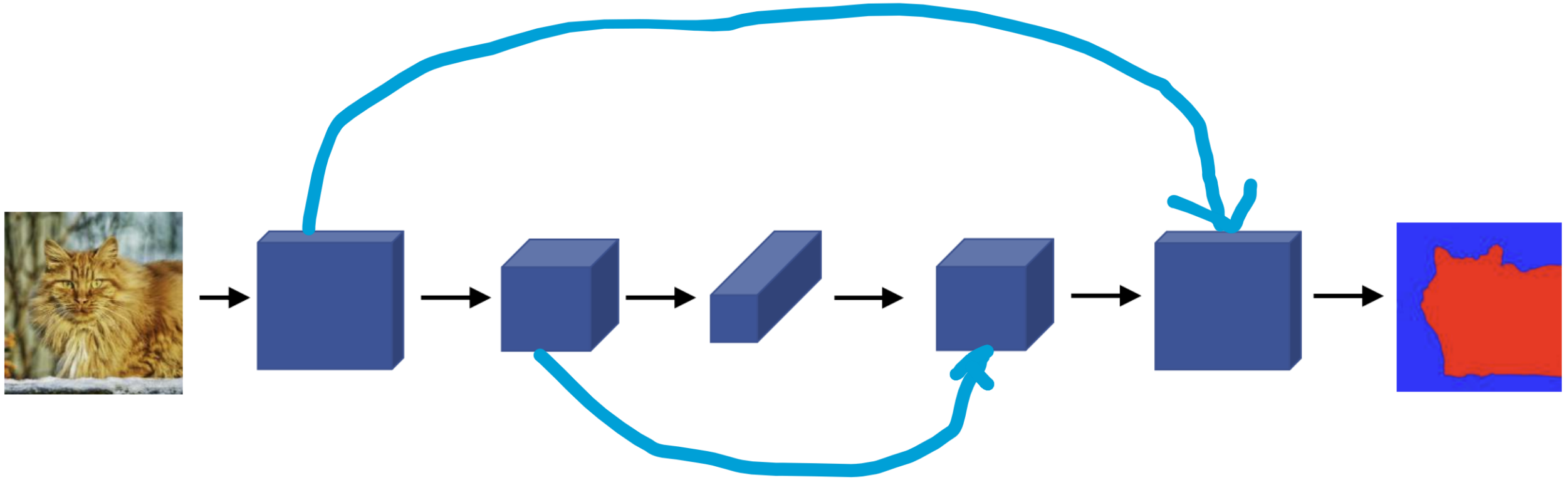
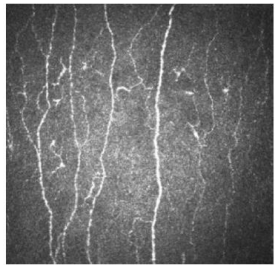
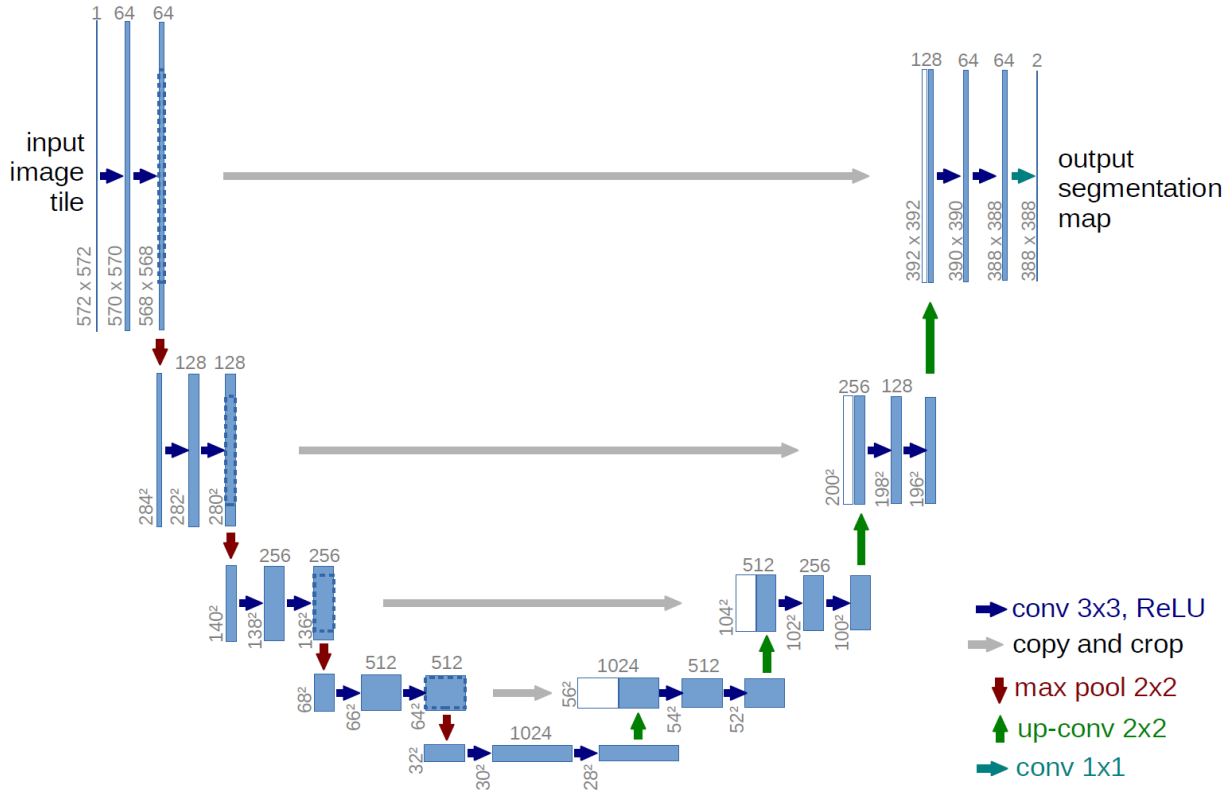
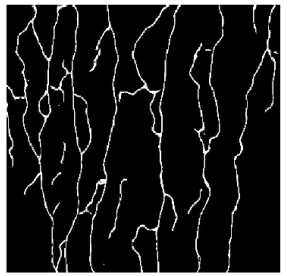


Image Segmentation

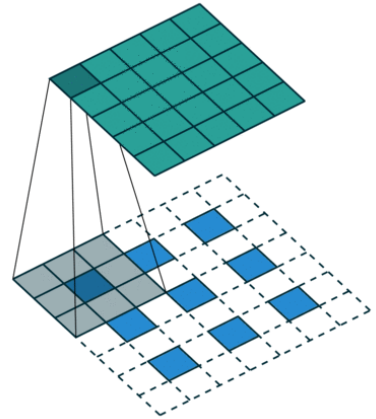
- Unet (74K citations): end to end image segmentation network
- Output has the same image resolution as the input
- Each pixel is considered as a classification problem







From 3x3 to 5x5



Fractionally strided convolution

O. Ronneberger et al. , U-Net: Convolutional Networks for Biomedical Image Segmentation, 2014

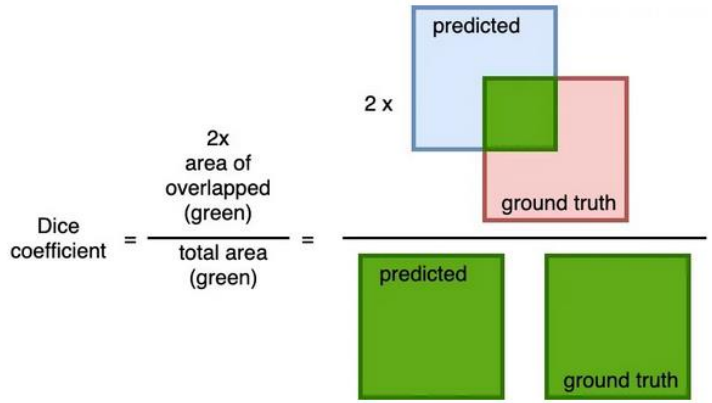
Loss Functions for Image Segmentation

- Cross Entropy and/or Dice Coefficient for Image Segmentation

$$L = -\sum_{n=1}^{N} \sum_{k=1}^{C} [y_{kn} \log(\hat{y}_{kn})]$$

$$L = 1 - \frac{2\sum_{\mathrm{p}} y\hat{y}}{\sum_{P} y^2 + \sum_{P} \hat{y}^2}$$

N: number of data samples
C: number of classes
$\hat{y}$: predictions
y: true labels
p: all pixels in an image



Other Well-known Methods

- Real-time object detection and recognition:
  YOLO (You only look once)
  Yolo8: https://yolov8.com/

- Real-time object detection and segmentation:
  Mask-RCNN https://arxiv.org/abs/1703.06870

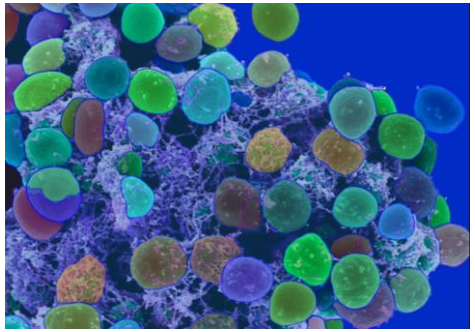
- Face recognition and facial expression recognition:
  DeepFace: DeepFace: Closing the Gap to Human-Level Performance in Face Verification - Meta Research (facebook.com)

- Segment Anything Model: https://segment-anything.com/

YOLO

Mask RCNN

DeepFace



SAM